Diagrama de Classe

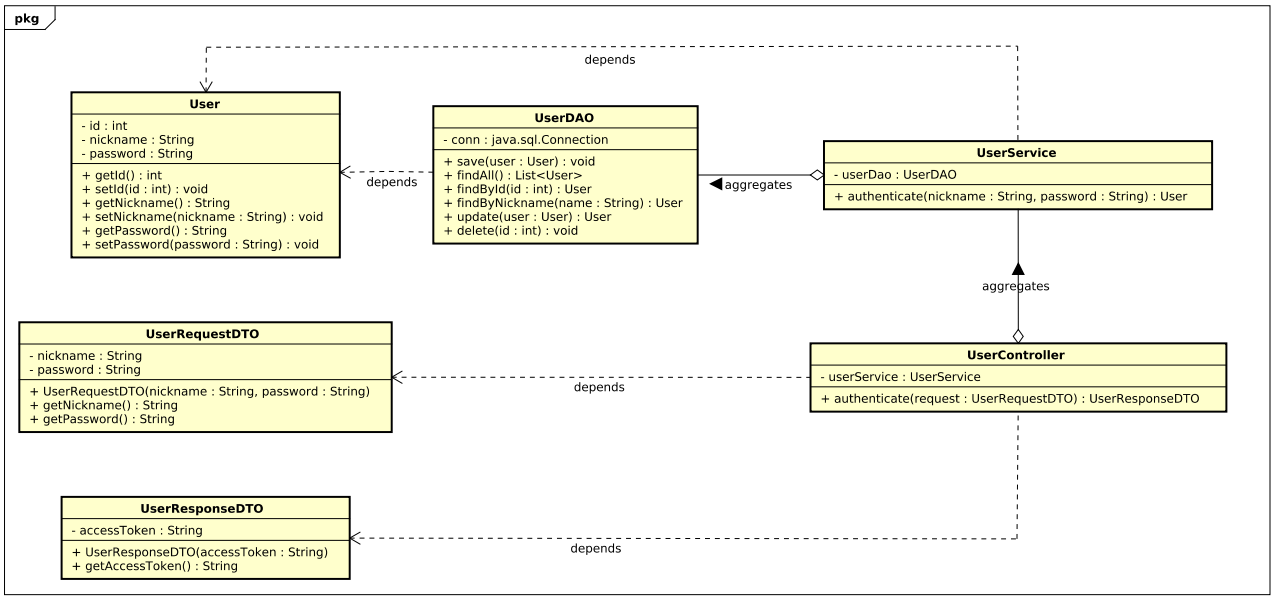
CSU01: Acesso do Aluno

Autor: Altamiro Santos Moura Belezia (Designer de Software)

Descrição do caso de uso:

Permite ao aluno criar uma nova conta no sistema ou acessar uma conta já existente para iniciar ou continuar sua experiência no jogo.

Diagrama:



Descrição do diagrama:

O sistema é dividido em camadas, cada uma com uma responsabilidade específica. No coração, temos a classe **User**, que representa a entidade do usuário com seus dados básicos, como ID, apelido (nickname) e senha.

Para gerenciar o acesso aos dados, existe a classe **UserDAO**. Pense nela como a ponte para o banco de dados. Ela é a única parte do sistema que sabe como buscar, salvar, atualizar e deletar informações de usuários.

Acima disso, está a classe **UserService**, que é a responsável pela lógica de negócio. Ela não interage diretamente com o banco de dados. Em vez disso, ela usa o **UserDAO** para realizar suas tarefas. Se você precisa registrar um novo usuário ou autenticar um usuário existente, a lógica para essas operações reside no **UserService**.

Por fim, a classe **UserController** atua como a interface do sistema. Ela recebe requisições externas (por exemplo, de um aplicativo ou de uma página web) e coordena a ação necessária. Se alguém

tenta fazer login, o **UserController** recebe os dados e os envia ao **UserService** para que a autenticação seja processada.

A classe **UserDTO** é um objeto simples usado para transferir dados, especialmente entre o **UserController** e o **UserService**. Ela garante que apenas os dados essenciais (como apelido e senha) sejam passados, protegendo informações internas como o ID do usuário.

Implementação:

Para construir esse sistema, siga uma abordagem em camadas:

1. **Crie a classe User**: Esta será a representação principal do seu usuário. Defina os atributos e os métodos para acessar e modificar esses dados.
2. **Implemente a camada de persistência**: Crie a classe **UserDAO**. Dentro dela, escreva os métodos para interagir com o banco de dados, usando uma conexão (conn: java.sql.Connection) para executar comandos de inserção, busca, atualização e exclusão.
3. **Desenvolva a lógica de negócio**: Crie a classe **UserService**. Essa classe deve ter uma referência para a classe UserDAO. Os métodos register e authenticate do UserService devem conter as regras de negócio do seu sistema, como a validação de senhas, a verificação de usuários duplicados e a criptografia.
4. **Construa a camada de controle**: Crie a classe **UserController**. Esta classe terá uma referência para a classe **UserService**. Os métodos register e authenticate aqui receberão os dados brutos da requisição. O **UserController** processa esses dados, os empacota em um objeto **UserRequestDTO**, e os repassa para o **UserService**.
5. **Utilize objetos de transferência de dados (DTOs)**:
    - Crie a classe **UserRequestDTO** para representar os dados de entrada vindos da requisição, garantindo que apenas as informações necessárias sejam recebidas.
    - Crie a classe **UserResponseDTO**, que conterá somente o atributo accessToken, sendo utilizado como resposta segura para operações de autenticação bem-sucedida.